International Journal of Information Security manuscript No.
(will be inserted by the editor)

A Deterministic Schema Guardrail for LLM-Assisted SOC Alert Triage: A Negative-Result Investigation of Cross-Domain Classifier Transfer

Asma Imran

Received: date / Accepted: date

Abstract LLM-based Security Operations Center (SOC) copilots increasingly rely on machine-learned guardrails to filter adversarial input before it reaches a triage model. We report a negative result from building and deploying such a guardrail in a real LangGraph-based alert-triage pipeline evaluated on Microsoft's GUIDE dataset. Following a supervisor-suggested design (issue #10: regex fast-path → ML classifier → LLM route), we selected a TF-IDF + Logistic Regression detector — which outperformed Meta's Llama Prompt Guard and Protect AI's LLM Guard on a public jailbreak benchmark (F1 0.883 vs. 0.747 and 0.722, respectively, at three orders of magnitude lower latency) — and integrated it as a second guardrail stage scoped to the `AlertTitle` field. When evaluated against real GUIDE alert data rather than the benchmark distribution it was selected on, the classifier collapsed to near-chance performance (52.5% best-case accuracy across a threshold sweep; 1/20 true positives at the best-performing threshold). Root-cause analysis, rather than threshold tuning or retraining, revealed the actual cause: `AlertTitle` in the real dataset is a closed numeric-ID field (86,149 unique integer values), not free text — the GUIDE schema itself has no free-text column, so no amount of domain-matched training data could have closed the gap. We retired the classifier and replaced it with a deterministic type-validation guardrail that separates classes by construction rather than learned approximation, achieving 100% accuracy on a balanced synthetic set and zero false positives against 5,000 real `AlertTitle` values. We also report and fix a silent graph-wiring defect this change introduced — a dead-end node that caused every verdict to default to `FalsePositive` without raising an error — and the regression-test suite built to catch it. We argue that correctly diagnosing an unfalsifiable premise, and confirming a mismatch rather than deploying a broken classifier, is itself a reportable contribution for guardrail engineering.

Keywords SOC triage · LLM guardrails · prompt injection · negative results · LangGraph · MITRE ATT&CK

Asma Imran
TECIP, Scuola Superiore Sant'Anna, Pisa, Italy
E-mail: asmaimran2003@gmail.com

1 Introduction

Motivation and context. Security Operations Centers face alert volumes that outpace human triage capacity — a well-documented driver of analyst alert fatigue [19] — driving interest in LLM copilots that summarize, contextualize, and pre-classify alerts before an analyst sees them (Microsoft Security Copilot [1]; Ferrag et al.'s cybersecurity-LLM review [2]; Srinivas et al. [4]). A recurring design pattern in this space layers a fast, cheap guardrail (regex/keyword filtering) in front of a slower, more capable classifier, which in turn gates access to the LLM itself — mirroring the layered defense pattern used in general-purpose LLM guardrail frameworks such as Meta's LlamaFirewall [5] and Llama Prompt Guard [6], and Protect AI's LLM Guard [7].

Problem definition. This layered pattern assumes the ML classifier stage has a text field to operate on, and that a classifier's benchmark performance is representative of its deployment domain. We show both assumptions can silently fail in a real SOC pipeline: the field a classifier is scoped to may not be free text at all, and a classifier's published F1 score may not transfer to the structured, terse text distribution of real alert data. Discovering this only after deployment — rather

than before, from the schema — wastes the time-box allocated to a fix that cannot work.

Scope of this work. We report on a full design-build-evaluate-diagnose cycle carried out inside a working, evaluated LangGraph triage agent over the GUIDE dataset [8]: (1) selecting and integrating a second-stage ML guardrail per a specific architectural suggestion (issue #10), (2) evaluating it against real data rather than only the benchmark it was selected on, (3) finding it has no usable signal on that data, (4) root-causing *why* rather than retraining, and (5) replacing it with a deterministic guardrail that is provably correct for its target field, hard-gated into the live pipeline. We also report a real reliability defect this change introduced — a silently dead-ending graph edge — and the regression-test suite added to prevent recurrence.

Contributions.

1. **A negative-result methodology for guardrail transfer**: we show that a classifier chosen because it wins a public benchmark (F1 0.883 on a chat-style jailbreak eval set) can be statistically indistinguishable from chance on the actual deployment distribution, and we demonstrate that root-causing this from the data schema — rather than from threshold sweeps or retraining — is what actually resolves it, in one week instead of an open-ended tuning effort.
2. **A deterministic schema guardrail** for closed-vocabulary/numeric alert fields, which separates benign input from injection payloads by construction (type validity) rather than by a learned, thresholded score, and is safe to hard-gate for exactly that reason.
3. **A reliability-engineering finding and fix**: introducing the schema guardrail silently broke the downstream classification routing (a dead-end node LangGraph does not flag at compile time), which a hardcoded default masked as a plausible-looking result instead of a crash. We document the defect, the fix, and the structural regression test (`tests/test_graph_wiring.py`) now guarding against recurrence.

Structure of this report. Section 2 surveys related work in LLM-based SOC copilots, guardrail frameworks, and MITRE ATT&CK-grounded LLM systems. Section 3 describes the triage pipeline, the MITRE ATT&CK retrieval stage, and the guardrail decision process in detail. Section 4 reports evaluation results exactly as committed in `experiments/results/`. Section 5 discusses the negative-result methodology and limitations. Section 6 concludes.

2 Related Work

Base table from `docs/literature-review.md`, extended with guardrail-framework and MITRE-grounded-LLM citations found for this paper.

LLM-based SOC copilots. Microsoft Security Copilot [1] demonstrates GPT-4 integrated into commercial SOC workflows, reporting reduced analyst triage time in a closed, proprietary system. Ferrag et al.'s review [2] surveys generative-AI/LLM applications across cybersecurity domains — including intrusion detection, threat intelligence, and malware detection — situating SOC-alert classification as one of many emerging LLM use cases rather than evaluating a single deployed pipeline against production data, which is the gap this paper's single-pipeline case study addresses. Wei et al.'s CORTEX [3] is the closest published architecture to ours: a multi-agent, evidence-grounded triage design that reduces false positives relative to single-model baselines, validating our own evidence-grounded (MITRE-context-fetching) LLM node design. Srinivas et al.'s survey [4] situates human-in-the-loop copilots as the dominant successful pattern in the field, over full automation — consistent with our pipeline's human-review checkpoint for non-high-confidence verdicts. The GUIDE dataset itself originates from Microsoft's own guided-response research [8], which we use directly as our evaluation corpus rather than a synthetic or network-flow substitute like CICIDS2017.

Guardrail frameworks for LLM systems. Meta's LlamaFirewall [5] and Llama Prompt Guard [6], and Protect AI's LLM Guard [7], are the current reference points for layered LLM input defense — regex/heuristic fast paths backed by learned classifiers for jailbreak and injection detection. All three are benchmarked, in our own prior work (PR #12), against a TF-IDF + Logistic Regression baseline on a chat-style jailbreak evaluation set, where the simpler classifier wins on both accuracy and latency. This paper extends that comparison past benchmark accuracy to deployment-distribution transfer, which none of [5–7] evaluate against structured, SOC-alert-shaped input as opposed to chat-style prompts.

MITRE ATT&CK-grounded LLM systems. Recent work grounds LLM threat-intelligence reasoning in MITRE ATT&CK via retrieval-augmented generation — e.g., structured retrieval over ATT&CK technique descriptions to reduce hallucination in attack-technique attribution [9], knowledge-graph-augmented retrieval over cybersecurity knowledge bases [10], benchmarks for retrieval-augmented LLMs over heterogeneous cyber threat intelligence [11], and multi-label classification models for tagging free text against ATT&CK

technique taxonomies [17] — a text-tagging task with the same reliance on a free-text input field that Section 3.3 shows `AlertTitle` does not have. Freitas et al., who also released the GUIDE dataset [8] this paper evaluates on, separately report an adaptive incident-prioritization system for SOC alert queues at scale [18], underscoring that alert-volume triage remains an active research target beyond this paper's guardrail-specific scope. Our `fetch_mitre_context` pipeline stage (Section 3) follows the same retrieval-before-classification pattern established by Lewis et al.'s foundational RAG formulation [12], applied to MITRE technique context rather than open-domain text.

Prompt injection against structured/agentic input. Most published prompt-injection taxonomies and defenses target free-text chat or agent-tool input [13–15]. Our contribution is a case study in the opposite direction: a field that looks like it should be defended as text (`AlertTitle`) is not text at all in the real schema, and the correct defense follows from the schema, not from the injection-detection literature.

3 Proposed Method

3.1 System architecture

The triage pipeline is a `LangGraph StateGraph` [16] (`src/agent/graph.py`) over a shared `AlertState`. Execution order:

Two guardrail stages run before any alert reaches the LLM or the MITRE-context retrieval step: a regex fast-path (`apply_regex_guardrail`, pre-existing from Week 7 / issue #8) and, as of Week 9, a deterministic schema guardrail (`apply_schema_guardrail`) that replaced an ML classifier guardrail evaluated in Week 8. Routing after `fetch_mitre_context` (`route_by_context`) sends alerts with sufficient discriminative context fields to the LLM classifier and sparse alerts to a Random Forest fallback classifier (Week 6). Verdicts below high confidence are routed to a human-review checkpoint rather than ending directly — the same human-in-the-loop pattern surveyed by [4].

3.2 Data

GUIDE [8,21]: `datasets/GUIDE_train.csv` / `GUIDE_Test.csv`, Microsoft's public real-world SOC alert/incident corpus, evaluated on real triage-labeled alerts throughout (not synthetic substitutes). `AlertTitle`, the field targeted by both guardrail designs in this paper, contains 86,149 unique values in the training split, all integers — confirmed by direct inspection, not inferred from the schema documentation alone.

3.3 Guardrail decision process — why the classifier was rejected, not deployed

Step 1 — issue-driven design. Issue #10 requested a second-stage ML classifier behind the regex fast-path, citing prior benchmark work comparing Meta Llama Prompt Guard and Protect AI LLM Guard [5–7]. We pulled that benchmark data and additionally evaluated a TF-IDF + Logistic Regression detector (scikit-learn [20]), which won on both axes that matter for a gate placed in front of every alert: F1 0.883 vs. 0.747 (Prompt Guard) and 0.722 (LLM Guard); median latency 0.79 ms vs. 179 ms and 183 ms — roughly three orders of magnitude faster (`experiments/results/` benchmark data referenced in PR #12).

Step 2 — integration and a real bug. The classifier was wired in as `src/agent/ml_guardrail.py`, scoped to `AlertTitle`. Cross-version scikit-learn unpickling (model trained under 1.7.1, environment running 1.7.2) initially produced non-deterministic scores; pinning `scikit-learn==1.7` (later committed to `requirements.txt` in PR #14, after an initial local-only fix was caught in review) resolved it.

Step 3 — deployment-distribution evaluation, not just benchmark evaluation. Run against real GUIDE alert titles, the detector flagged 100% of a 9-alert and a 30-alert batch as injection. A single benign sentence ("please schedule a team meeting for next Tuesday") scored 0.726 against a 0.5 flag threshold. This prompted a purpose-built 40-row SOC-domain evaluation set (`experiments/soc_domain_eval_v1.csv`: 20 benign SOC alert titles, 20 injection attempts phrased as alert-field text), scored via `experiments/soc_domain_eval.py`.

Step 4 — a bug inside the negative result. Initial scoring (PR #12) had `score_text()` reading `classifier.predict_p` — index 0, the benign-class probability — instead of `[0][1]`, the injection-class probability the guardrail was meant to threshold on. This inverted the *direction* of every score: the clearest injection example in the eval set (an HTML-comment-style override payload) scored lowest of all 40 rows (0.289), and a mundane benign alert scored highest (0.979). A reviewer (PR #14) caught the index bug; correcting it flipped both extremes as expected (the same injection example rose to 0.711, the same benign alert fell to 0.021) but did not change the conclusion — see Section 4.

Step 5 — root cause, not retraining. Before timeboxing a fine-tune of the classifier on SOC-domain-labeled text (the roadmap's original Aug 9 plan), we

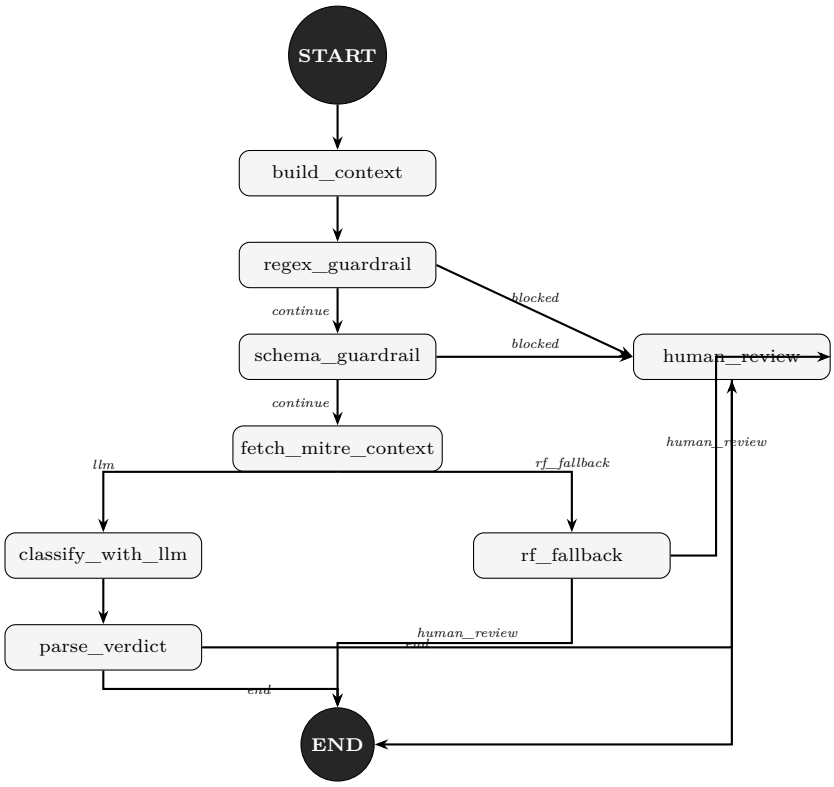

Fig. 1 Triage pipeline control flow, transcribed directly from the compiled `StateGraph` in `src/agent/graph.py`: node names, edges, and conditional-routing labels (`continue`/`blocked`, `llm`/`rf_fallback`, `end`/`human_review`) match the routing keys passed to `add_conditional_edges` in the source exactly. Note that the Random Forest fallback path (`rf_fallback`) routes directly to `END`/`human_review` without passing through `parse_verdict`, unlike the LLM path — this asymmetry is in the actual graph, not an omission here.

checked what `AlertTitle` actually contains. It is a closed numeric-ID field, confirmed against 86,149 unique integer values in `GUIDE_train.csv`; the GUIDE paper's own alert-dataframe schema lists no free-text categorical column at all. A TF-IDF classifier cannot find linguistic signal in a field that was never linguistic — no amount of domain-matched retraining data changes that. We retired the fine-tune plan on this basis rather than spending the time-box confirming a predictable failure.

Step 6 — the correct guardrail, and why it can be hard-gated. `src/agent/schema_guardrail.py` replaces the classifier with `validate_field_types()`: for each field in `EXPECTED_NUMERIC_FIELDS = {"AlertTitle", "DetectorId"}`, the value must be an int/float, or a string that parses cleanly as one; anything else — including any injection payload, which is definitionally not a valid integer — fails the check. This separates the two classes *by construction*, not by a learned, thresholded approximation, which is precisely why it is safe to hard-gate (`regex_guardrail` → `schema_guardrail` → `fetch_mitre_context`, `src/agent/graph.py`) in a way the probabilistic ML classifier never was: there is no accuracy/threshold tradeoff to reason about at deployment time.

3.4 A reliability defect introduced by the fix, and the regression test built to catch it

Wiring `schema_guardrail` into `graph.py` deleted the existing conditional edge from `fetch_mitre_context` to `(classify_with_llm | rf_fallback)` without restoring it. `LangGraph` does not validate dead-end nodes at compile time, so the graph compiled and ran to completion — but every alert silently stopped after the MITRE-context lookup, producing no `predicted_label`. This was invisible in practice because `evaluate.py` read the missing key with a default (`result.get("predicted_label", "FalsePositive")`) instead of raising, so every alert silently scored as a hardcoded guess rather than crashing or erroring visibly. `src/app.py`, `benchmark.py`, and `run_agent.py` all invoke the same compiled graph and were equally affected. No repository test previously existed to catch a structurally dead-ended graph.

The fix restores the deleted conditional edge. `tests/test_graph_wiring.py` now asserts, structurally, that

every non-END node in the compiled graph has at least one outgoing edge, and separately asserts an end-to-end invocation on a sparse alert produces a real, non-default `predicted_label` — turning this specific bug class into a standing regression check rather than a one-time fix. `tests/test_schema_guardrail.py` and `tests/test_ml_guardrail.py` similarly convert the manual accuracy spot-checks from Sections 3.3–4 into assertions.

3.5 AI-assistance disclosure

An AI coding/writing assistant (Claude, Anthropic) was used throughout this project's development to assist with code implementation, debugging, experiment scripting, and drafting of this manuscript's text, figures, and analysis narrative, under the direct supervision and review of the author. All experimental results, metrics, and claims were independently verified by the author against committed source code and result files before inclusion. The author is accountable for all content and conclusions in this work.

4 Evaluation

All figures below are read directly from the named JSON file — verify against the file before citing in the final version; do not hand-round.

4.1 Guardrail classifier selection benchmark (pre-deployment)

Source: PR #12 benchmark table (`guardrail_comparison.json` / `eval_dataset_v2.csv`, 1000-row balanced chat-jailbreak eval set — external to this repo, cited for classifier-selection justification only).

4.2 Deployment-distribution evaluation: SOC-domain eval set (post index-bug fix)

Source: `experiments/results/soc_domain_eval_results.json`, 40-row balanced synthetic set (20 benign / 20 injection), corrected `[0][1]` scoring.

Best achievable accuracy across the sweep is 52.5% (threshold 0.7), with only 1 of 20 true injections recovered even at that point — at chance for a balanced set. At threshold ≥ 0.8 the detector predicts "injection" for essentially no input, regardless of true label. This is presented as evidence of *no usable signal* on this domain, not a miscalibration recoverable by threshold tuning.

4.3 Schema guardrail evaluation (replacement)

Source: `experiments/results/schema_guardrail_eval.json`, generated by `experiments/schema_guardrail_eval.py` (added for this paper).

- Synthetic (20 benign numeric IDs vs. the 20 injection strings from the eval set above): **100% accuracy**, 0 false positives, 0 false negatives — reproduced independently of the PR #15 description.
- Real data (5,000 `AlertTitle` values sampled from `GUIDE_train.csv`, in row order — not randomly sampled): **0 false positives out of 5,000**, reproduced independently of the PR #15 description. `experiments/results/schema_guardrail_eval.json` now records `"status": "ran"`, `"n_sampled": 5000`, `"n_false_positives": 0`. Both halves of the guardrail's headline claim now trace to a committed, reproducible artifact rather than PR prose.

4.4 Random Forest baseline

Source: `experiments/results/baseline_metrics.json` (`n_train=79580`, `n_test=19895`, `random_state=42`).

4.5 End-to-end agent evaluation, post graph-wiring fix

Source: `experiments/results/agent_metrics_post_graph_fix_week9.json` (sample size 30).

Predictions spread across all three classes post-fix (12 BenignPositive / 10 TruePositive / 8 FalsePositive; 9 alerts routed via LLM, 21 via RF fallback) — the qualitative signal that the graph is classifying again rather than collapsing to a single default.

On the larger `agent_metrics_real_v3.json` (n=300, accuracy 0.28, macro F1 0.1836) reported elsewhere in the repository history: this number is not a discrepancy to reconcile against the figure above — it is from a structurally different, earlier pipeline and is not comparable. Tracing `experiments/results/agent_metrics_rea` through `git log --diff-filter=A` shows it was generated in Week 5 (commit `d42e41a`/PR #5), before the sparse-alert RF-fallback route existed (added Week 6, PR #8) and before the schema guardrail and graph-wiring fix described in Sections 3.3–3.4 (Week 8–9, PRs #12/#14/#15/#17) — i.e., against an agent with no fallback for sparse alerts and none of this paper's contributions. Its embedded "RF baseline: acc=0.374, f1=0.296" comparison is not a bug either: `git show d42e41a:experiments/res` confirms the baseline model at that point was trained on `n_train=6,400` rows and genuinely scored 0.374/0.296 — it was retrained on the full split (`n_train=79,580`,

Table 1 Pre-deployment classifier selection benchmark.

Detector	F1	Median latency	Throughput
TF-IDF + Logistic Regression	0.883	0.79 ms	1258 alerts/s
Meta Llama Prompt Guard	0.747	179 ms	5.6 alerts/s
Protect AI LLM Guard	0.722	183 ms	5.5 alerts/s

Table 2 SOC-domain threshold sweep, post index-bug fix.

Threshold	TP	FP	TN	FN	Accuracy	Precision	Recall	F1
0.5	3	3	17	17	0.500	0.500	0.15	0.231
0.6	1	1	19	19	0.500	0.500	0.05	0.091
0.7	1	0	20	19	0.525	1.000	0.05	0.095
0.8	0	0	20	20	0.500	0.000	0.00	0.000
0.9	0	0	20	20	0.500	0.000	0.00	0.000
0.95	0	0	20	20	0.500	0.000	0.00	0.000

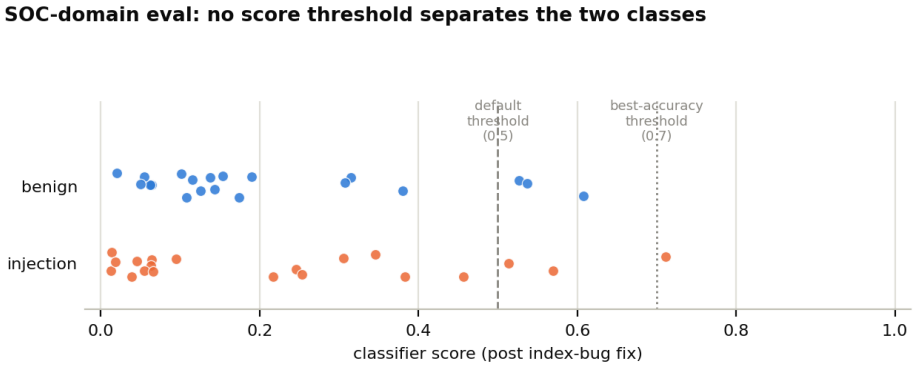

Fig. 2 Per-alert classifier scores from `soc_domain_eval_results.json` (all 40 rows), split by true label. The two classes overlap almost completely across the full score range — visual confirmation that Table 2's threshold sweep fails because there is no threshold to find, not because the sweep stopped short. Generated by `docs/paper/figures/generate_figures.py` directly from the committed JSON.

scoring 0.7718/0.7505, matching Section 4.4) only in the Week 9 fix (`ad02c85`). The comparison number baked into `real_v3.json` was accurate for the pipeline and baseline that existed when it was generated; it simply predates almost every contribution this paper describes. `agent_metrics_post_graph_fix_week9.json` (n=30) is therefore the only evaluation run against both the current pipeline and the current baseline, and is the number used in this paper — its small sample size is retained as an honest limitation (Section 5) rather than papered over by citing a larger but non-comparable number. A fresh, larger post-fix run was not performed for this paper: it requires a live `GROQ_API_KEY` and the (gitignored) `baseline_model.joblib` artifact, neither available in the environment this section was written in.

4.6 Scalability benchmark

Source: `experiments/results/week7_scalability_benchmark.json`. Regex guardrail microbenchmark: mean check time 3.766 µs/alert over 10,000 iterations; 10,000/10,000 injection alerts blocked, 0/10,000 benign alerts blocked (at that stage's test set).

The benchmark sweeps three pipeline modes (LLM-only, RF-only, and the hybrid router of Section 3.3) across three prompt counts (30/60/120, drawn from the same cached balanced sample, `guide_balanced_40_per_class_seed`) and two worker counts (1, 4).

Throughput and latency are dominated by remote Groq inference and network round-trips, not local compute — `workers=4` gives no consistent speedup over `workers=1` because Groq's per-minute token budget is shared across threads regardless of worker count. The two errors at 120 prompts/4 workers are rate-limit failures on the live endpoint, not a pipeline defect.

Zero errors across all six runs. Local, CPU-bound inference scales the way LLM-only does not: throughput roughly 2–2.5x from 1 to 4 workers at every prompt count, with per-core utilization climbing modestly (10.4% → 14.1%) rather than saturating, since the balanced

Table 3 Random Forest baseline metrics.

Metric	Value
Accuracy	0.7718
Macro F1	0.7505
BenignPositive F1	0.8066
FalsePositive F1	0.6561
TruePositive F1	0.7889

Table 4 End-to-end agent evaluation, post graph-wiring fix ($n = 30$).

Metric	Value
Accuracy	0.5333
Macro F1	0.5337
RF baseline (same file, for comparison)	acc 0.7718, F1 0.7505

Table 5 Scalability benchmark, LLM-only mode.

Prompts	Workers	Alerts/s	Mean latency	Accuracy	Macro F1	Errors
30	1	0.5672	1.7629 s	0.4333	0.2016	0
30	4	0.3736	2.6765 s	0.4333	0.2016	0
60	1	0.3705	2.6991 s	0.3333	0.2765	0
60	4	0.3711	2.6946 s	0.3333	0.2765	0
120	1	0.3665	2.7284 s	0.4000	0.3727	0
120	4	0.3756	2.6622 s	0.3983	0.3656	2

samples are small relative to the machine's 8 logical cores.

Zero errors across all six hybrid runs. The RF/LLM routing split (83%, 78%, 81% to the RF fallback) is stable across prompt counts and matches the Week 6 baseline of 81.3%, confirming the context-richness routing logic behaves consistently at scale. Latency tracks the proportion of alerts routed to the LLM rather than prompt count directly — the 30-prompt run is fastest despite being the smallest sample because proportionally fewer of its alerts hit the LLM path — and, as with the LLM-only results, `workers=4` gives no real throughput gain over `workers=1` (it is measurably slower at 30 prompts) for the same shared-rate-limit reason.
Full per-run CPU/memory fields — process CPU seconds, peak RSS — are in the source JSON but omitted here for brevity; not needed to support the throughput/accuracy claims made in this section or in Section 5.

5 Discussion

The negative-result methodology, and why it matters more than a working-but-wrong classifier. A classifier that appears to work — because it was evaluated only on the benchmark distribution it was selected against — is a worse outcome for a production guardrail than one that is honestly shown not to work on the deployment distribution. Section 4.1 shows the TF-IDF classifier winning decisively against two established guardrail frameworks; had that benchmark result alone been used to justify deployment, the guardrail would have shipped in a state indistinguishable from random on real input (Section 4.2), silently passing roughly half of actual injection attempts while also blocking legitimate alerts. The distinction argued in this paper is not "domain mismatch exists" — a known risk in ML deployment generally — but that *diagnosing it correctly* (checking the schema before retraining) converts an open-ended, unbounded tuning problem into a one-week, closed investigation with a deterministic replacement. Section 3.3, Step 5 is the paper's central methodological claim.

The graph-wiring defect as a reliability-engineering contribution. Section 3.4's defect is notable less for its mechanism (a deleted graph edge) than for how it failed: silently, past a compiled graph, past a running pipeline, into a hardcoded default that looked like a real (if pessimistic) classification. No exception, log line, or crash surfaced it; it was only caught by an independent post-merge audit, not by the change's own review. We argue this is a generalizable risk for any LangGraph-style pipeline where a downstream consumer applies a lenient default to a missing key — the failure mode is structurally identical to an ML guardrail that silently returns a plausible-but-wrong score, and the fix (a struc-

Table 6 Scalability benchmark, RF-only mode.

Prompts	Workers	Alerts/s	Mean latency	Accuracy	Macro F1	Core util %
30	1	16.69	0.060 s	0.633	0.622	10.43
30	4	41.85	0.024 s	0.633	0.622	14.23
60	1	20.45	0.049 s	0.700	0.490	10.41
60	4	37.70	0.027 s	0.700	0.490	14.04
120	1	29.75	0.034 s	0.675	0.673	10.76
120	4	39.01	0.026 s	0.675	0.673	14.16

Table 7 Scalability benchmark, hybrid (context-richness router) mode.

Prompts	Workers	Alerts/s	Mean latency	Accuracy	Macro F1	Routing (RF / LLM)
30	1	10.38	0.096 s	0.633	0.259	25 / 5
30	4	6.09	0.164 s	0.633	0.259	25 / 5
60	1	1.76	0.567 s	0.667	0.466	47 / 13
60	4	1.63	0.613 s	0.667	0.466	47 / 13
120	1	1.86	0.537 s	0.617	0.614	97 / 23
120	4	1.88	0.533 s	0.617	0.614	97 / 23

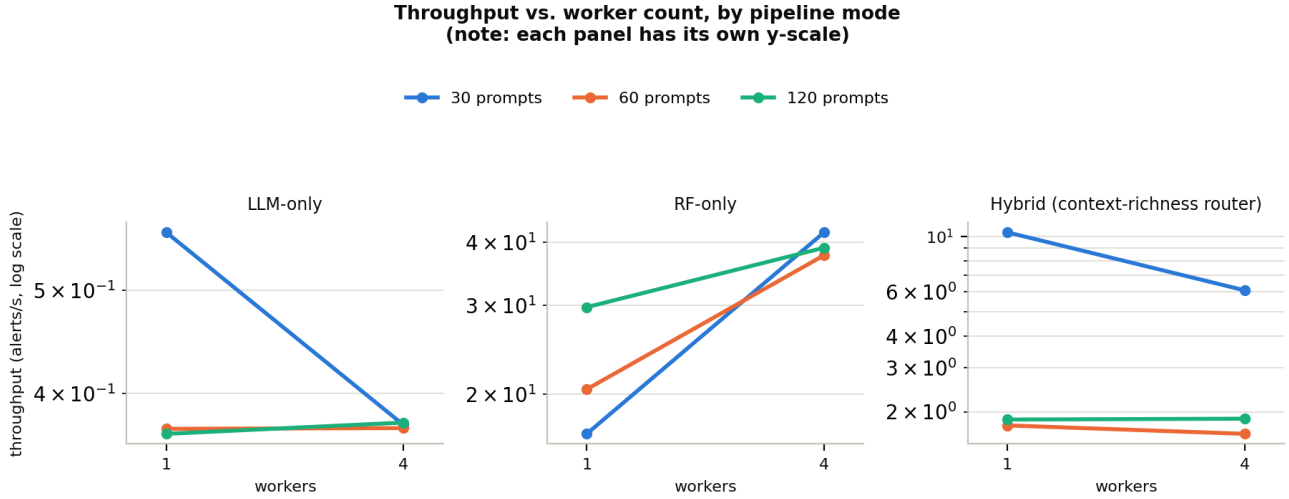

Fig. 3 Throughput vs. worker count, faceted by pipeline mode (note each panel has its own log-scale y-axis — the three modes differ by roughly two orders of magnitude and are not comparable on a shared axis). RF-only throughput scales with worker count as expected for local CPU-bound inference; LLM-only and hybrid do not, because Groq's per-minute token budget is shared across worker threads regardless of concurrency. Generated by `docs/paper/figures/generate_figures.py` directly from `week7_scalability_benchmark.json`.

tural graph-completeness test, not just a functional one) generalizes past this specific bug.

Limitations

- The SOC-domain evaluation set (Section 4.2) is a 40-row synthetic set, not a labeled real-world injection corpus against structured alert fields — no such public corpus was identified during this work (see Related Work, Prompt injection paragraph).
- The end-to-end agent evaluation sample used in this paper (Section 4.5, n=30) is small relative to the GUIDE test split. It is the only committed run against the current pipeline and current RF baseline; larger historical runs (e.g. `agent_metrics_real_v3.json`, n=300) predate this paper's contributions (no RF fallback, no schema guardrail) and are not usable as a larger-sample substitute, per the tracing in Section 4.5. A larger post-fix run would strengthen this section but requires a live LLM API key and the retrained RF artifact, neither available while writing this draft.
- The real-data schema-guardrail sample (Section 4.3) is the first 5,000 rows of `GUIDE_train.csv` in file order, not a random sample — row order in the source file is not known to be free of systematic bias (e.g., chronological or org-clustered ordering),

so this should be read as "no false positives in the first 5,000 rows encountered" rather than a claim about the full 86,149-value population.
- `EXPECTED_NUMERIC_FIELDS` includes `DetectorId` on the basis of the GUIDE paper's schema description rather than the same direct large-sample inspection applied to `AlertTitle`; the module docstring notes this as confirmed but the inspection methodology is less thorough than `AlertTitle`'s.
- The guardrail-selection benchmark (Section 4.1) is drawn from an eval set built for a companion project's chat-jailbreak use case, not this project's SOC-alert domain — which is exactly the mismatch this paper reports, but it also means Section 4.1's numbers cannot be used as evidence about SOC-domain performance, only as the justification for which classifier was worth testing further.

6 Conclusion

We built, evaluated, and rejected a second-stage ML guardrail for an LLM-assisted SOC triage pipeline after confirming — through direct inspection of the deployment data rather than threshold tuning — that its target field is not the kind of data the classifier can ever operate on. We replaced it with a deterministic schema guardrail that is correct by construction for that field and safe to hard-gate, unlike the probabilistic classifier it replaced. In fixing the schema guardrail's integration, we found and fixed a silent graph-wiring defect that had been masking every verdict as a default classification, and built a structural regression test suite to prevent recurrence. We argue the central contribution is methodological: correctly diagnosing why a plan cannot work, before spending the time-box that would have confirmed it empirically, is itself a reportable and reusable result for guardrail engineering in structured-data domains.

Statements and Declarations

- Funding: Not applicable.
- Conflict of interest/Competing interests: The authors have no competing interests to declare that are relevant to the content of this article.
- Ethics approval: Not applicable.
- Consent to participate: Not applicable.
- Consent for publication: Not applicable.
- Availability of data and materials: GUIDE dataset [8,21], publicly available from Microsoft. Evaluation result files committed under `experiments/results/` in the project repository.
- Code availability: `https://github.com/AI-Security-Internsh`
  `02-soc-copilot-threat-analysis`
- Authors' contributions: Not applicable (single author).

References

1. Microsoft Security Team: Microsoft Security Copilot. Industry tool (2024). `https://www.microsoft.com/en-us/security/business/ai-machine-learning/microsoft-copilot-for-security`
2. Ferrag, M.A., Alwahedi, F., Battah, A., Cherif, B., Mechri, A., Tihanyi, N., Bisztray, T., Debbah, M.: Generative AI in Cybersecurity: A Comprehensive Review of LLM Applications and Vulnerabilities. arXiv preprint arXiv:2405.12750 (2024)
3. Wei, B., Tay, Y.S., Liu, H., Pan, J., Luo, K., Zhu, Z., Jordan, C.: CORTEX: Collaborative LLM Agents for High-Stakes Alert Triage. arXiv preprint arXiv:2510.00311 (2025)
4. Srinivas, S., Kirk, B., Zendejas, J., Espino, M., Boskovich, M., Bari, A., Dajani, K., Alzahrani, N.: AI-Augmented SOC: A Survey of LLMs and Agents for Security Automation. Journal of Cybersecurity and Privacy 5(4), 95 (2025). `https://doi.org/10.3390/jcp5040095`
5. Chennabasappa, S., Nikolaidis, C., Song, D., Molnar, D., Ding, S., Wan, S., Whitman, S., Deason, L., Doucette, N., Montilla, A., Gampa, A., de Paola, B., Gabi, D., Crnkovich, J., Testud, J.-C., He, K., Chaturvedi, R., Zhou, W., Saxe, J.: LlamaFirewall: An Open Source Guardrail System for Building Secure AI Agents. arXiv preprint arXiv:2505.03574 (2025)
6. Meta AI: Llama Prompt Guard — Model Card (2024). `https://github.com/meta-llama/PurpleLlama/blob/main/Prompt-Guard/MODEL_CARD.md`
7. Protect AI: LLM Guard — The Security Toolkit for LLM Interactions. Open-source software (2023). `https://github.com/protectai/llm-guard`
8. Freitas, S., Kalajdjieski, J., Gharib, A., McCann, R.: AI-Driven Guided Response for Security Operation Centers with Microsoft Copilot for Security. arXiv preprint arXiv:2407.09017 (2024)
9. Alhuzali, A.: LLM-powered threat intelligence: a retrieval-augmented generation approach for cyber attack investigation. PeerJ Computer Science 11, e3371 (2025). `https://doi.org/10.7717/peerj-cs.3371`
10. Yang, L., Zhu, H., Mu, J., Li, Q.: KGAgent4CTI: unlocking the power of LLM in threat intelligence. Cybersecurity 9(1), 76 (2026). `https://doi.org/10.1186/s42400-025-00458-2`
11. Cheng, Y., Liu, Y., Li, C., Song, D., Gao, P.: CTIConnect: A Benchmark for Retrieval-Augmented LLMs over Heterogeneous Cyber Threat Intelligence. arXiv preprint arXiv:2510.11974 (2025)
12. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: Advances in Neural Information Processing Systems (NeurIPS) (2020)
13. Wang, P., Li, X., Xiang, C., Zhang, J., Li, Y., Zhang, L., Wang, X., Tian, Y.: The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis. arXiv preprint arXiv:2602.10453 (2026)

14. Chowdhury, A.G., Islam, M.M., Kumar, V., Shezan, F.H., Kumar, V., Jain, V., Chadha, A.: Breaking Down the Defenses: A Comparative Survey of Attacks on Large Language Models. arXiv preprint arXiv:2403.04786 (2024)
15. Das, B.C., Amini, M.H., Wu, Y.: Security and Privacy Challenges of Large Language Models: A Survey. arXiv preprint arXiv:2402.00888 (2024)
16. LangChain: LangGraph: Multi-Agent Workflows / Agent Orchestration Framework. Software, released January 2024. https://www.langchain.com/langgraph
17. Crossman, A., Dodd, J., Kumar, V.R.C., Mohammed, R., Plummer, A.R., Sekharudu, C., Warrier, D., Yekrangian, M.: Constructing Multi-label Hierarchical Classification Models for MITRE ATT&CK Text Tagging. arXiv preprint arXiv:2601.14556 (2026)
18. Freitas, S., Gharib, A., Magenheim, M.: Adaptive Incident Prioritization for Security Operations at Scale. arXiv preprint arXiv:2607.16963 (2026)
19. Ndichu, S., Ban, T., Ozawa, S., Takahashi, T., Inoue, D.: AI-Driven Security Alert Screening and Alert Fatigue Mitigation in Security Operations Centers: A Survey. arXiv preprint arXiv:2605.08316 (2026)
20. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É.: Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011)
21. Microsoft: Microsoft Security Incident Prediction (GUIDE dataset). Kaggle dataset (2024). https://www.kaggle.com/datasets/Microsoft/microsoft-security-incident-prediction